[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Introduction to Genetic Algorithms

University of Essex
School of Computer Science and Electronic Engineering

What did we learn last week? (1)

- We have learned about **combinatorial optimisation** problems (Knapsack problem & Bin Packing problem) and that **exact algorithms** can be **computationally complex** (especially intuitive solutions). Consequently, exact algorithms may not be a viable solution to real-world problems, and we need alternatives.
- **Meta-heuristic stochastic optimisation methods** such as **Evolutionary Algorithms** are such an alternative.
- In this module we will learn about EAs and work with them as tools to solve different types of problems.

What did we learn last week? (2)

- **Natural selection** means individuals in a population that are fitter with respect to their environment produce more offspring. The trait that made those individuals succeed becomes common in the population.
- **Genes** encode the various traits of an individual. All traits of an individual are combined in a **linear representation** (1-D array) (DNA).
- **Genetic recombination (crossover, XO)**, i.e., the exchange of genetic material to create offspring's that are different from the parents, and **random mutation** drive evolution.

Today’s learning objectives (Part 1)

- Understand the **nature-to-computer** mapping
- See how all Evolutionary Algorithms (EAs) conform to a **General Evolutionary** (GEA)
- Become familiar with a **binary generational Genetic Algorithm** (GA)

Evolutionary Algorithms (EAs)

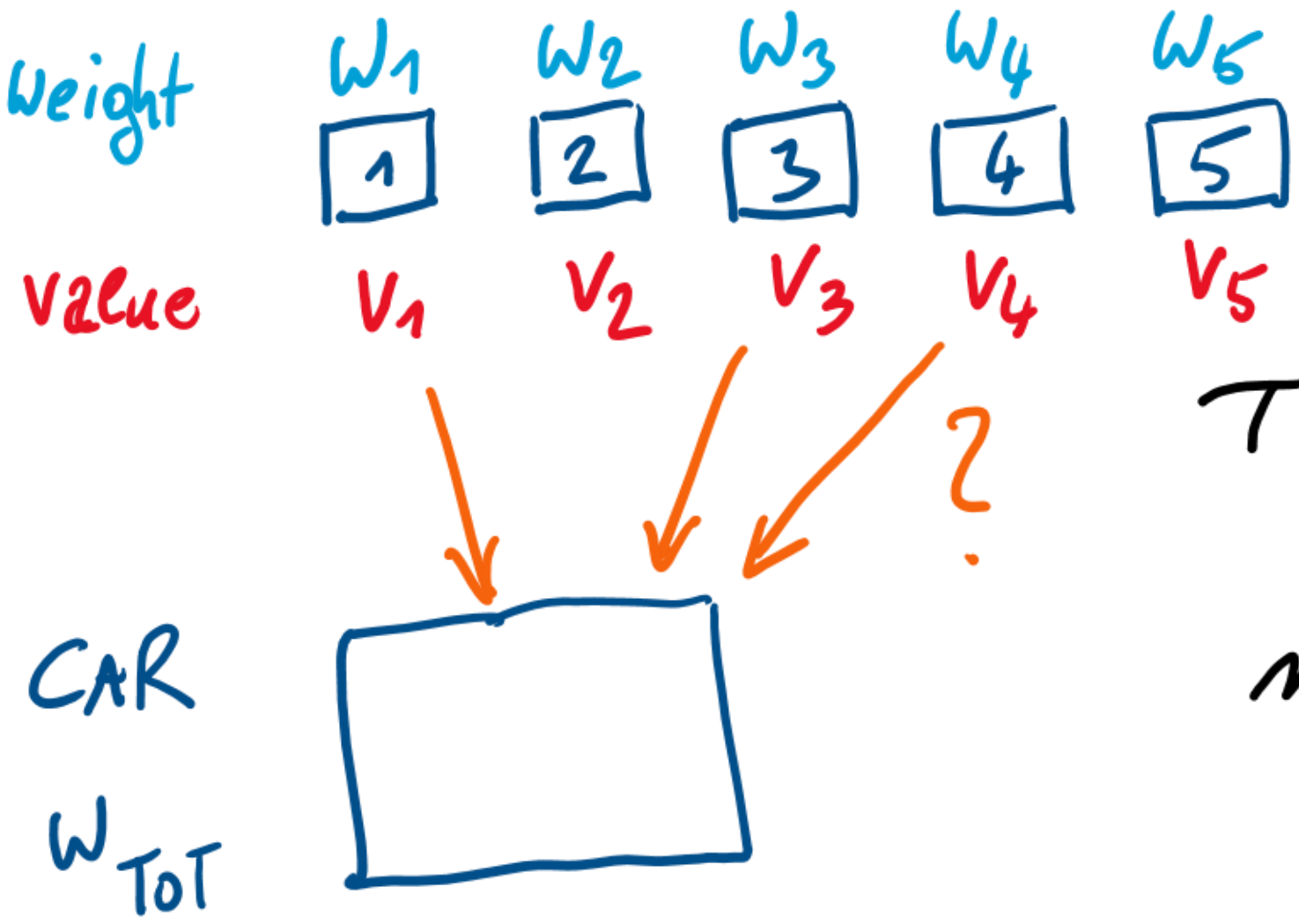
- EAs are algorithms **inspired to natural evolution**, i.e., EAs simulate the key ingredients of evolution.
- Applications:
  - Combinatorial optimisation (e.g., which packages go in a lorry to maximise load value? What is the shortest route to deliver packages?)
  - Function optimisation (e.g., minimum or maximum of a function)
  - Automatic programming (GP)
  - Machine learning
  - Problem solving
  - Automatic design, etc.

Nature-to-Computer mapping

- **Individual**: Solution to a problem
- **Population**: Set of solutions
- **Fitness**: Quality of a solution
- **Chromosome**: Representation for a solution
- **Gene**:Part of the representation of a solution
- **Growth**: Decoding of solutions (from “genes” to “creatures” :-)
- **Crossover and mutation**: Search operators
- **Natural selection**: Reuse of good (sub-)solutions

Example: Knapsack (Rucksack) problem

- Problem you face if you are constrained by a fixed-size knapsack and must fill it with the most valuable items.
- Given a **set of items** (i=1…N), each with a **weight/size** (w_i) and a **value** (v_i), determine which items to include in a **collection** so that the **total weight is less than or equal to a given limit/capacity** (w_{max}) and the **total value is as large as possible** (***maximize*** $\mathbf{v_{tot}}$).



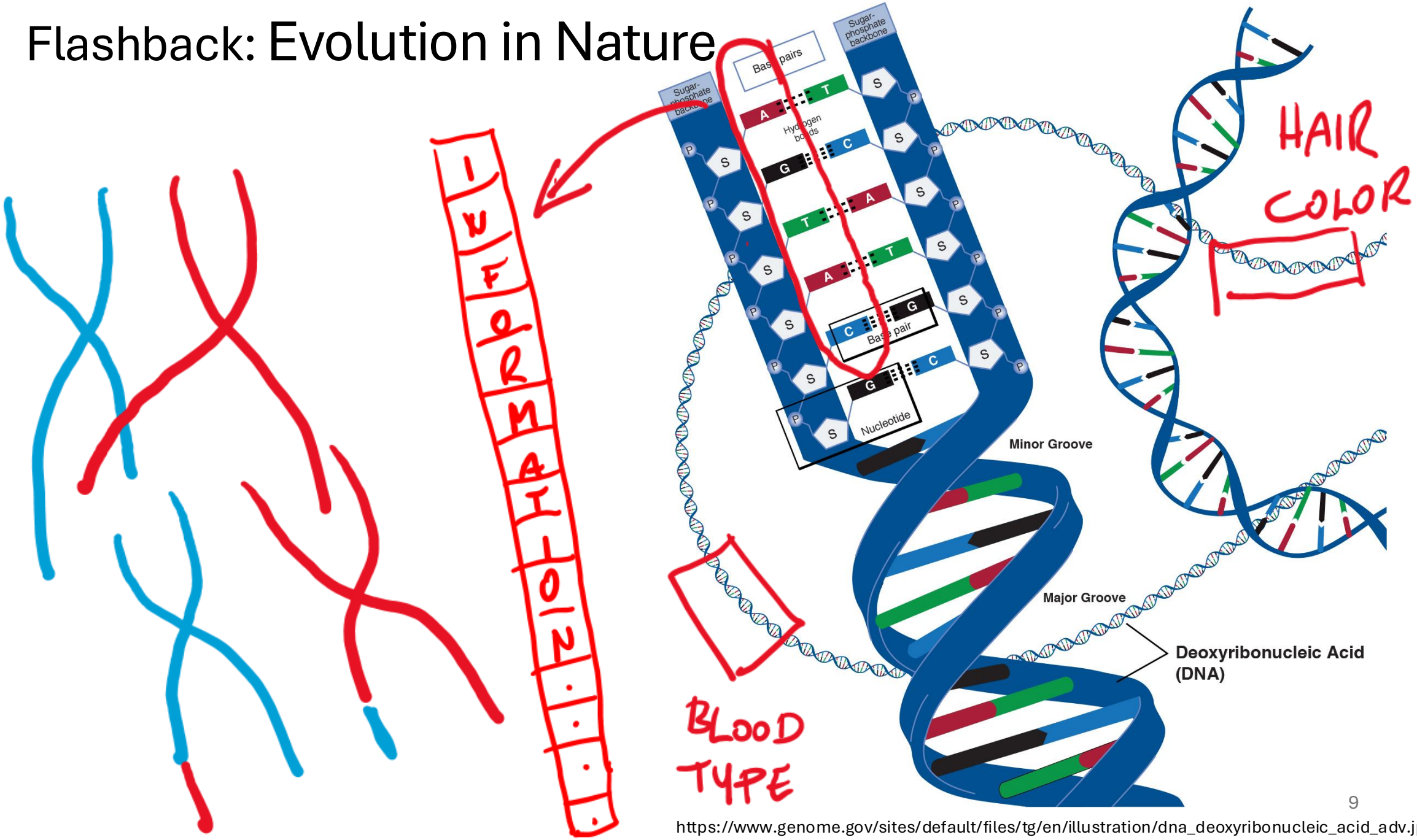
$n = 5$ boxes

$T = \{\text{chosen items}\}$

$$\text{maximise} \sum_{i \in T} v_i$$

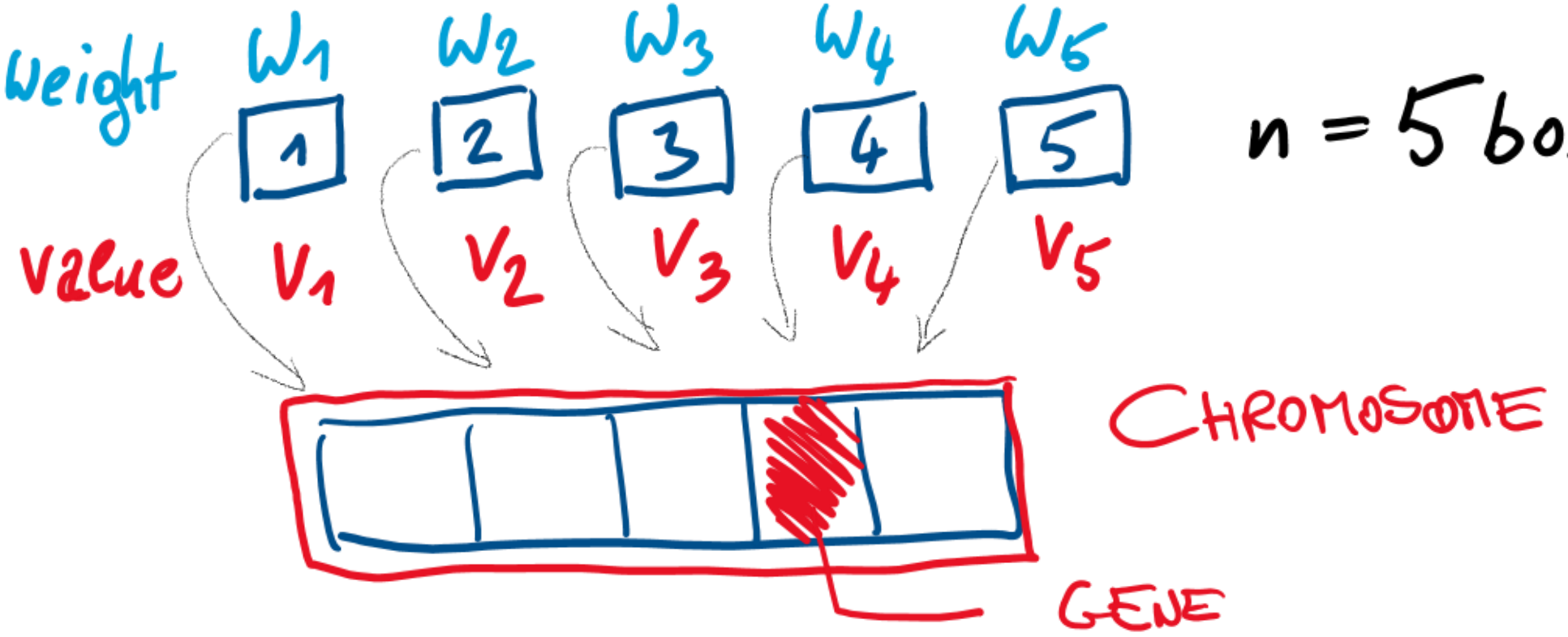
$$\text{subject to} \sum_{i \in T} w_i \leq W_{TOT}$$

Flashback: Evolution in Nature



https://www.genome.gov/sites/default/files/tg/en/illustration/dna_deoxyribonucleic_acid_adv.jpg

Knapsack: Representation of the solution?



$n = 5$ boxes

1 ... LOAD BOX INTO CAR

0 ... DO NOT LOAD BOX INTO CAR

Knapsack: Fitness of the solution?

	1	2	3	4	5
	2	3	5	5	3
	9	7	5	4	4

0	1	1	0	1

GROW ↓

$T = \{2, 3, 5\}$

FITNESS f

$$\sum_{i \in T} w_i = 3+5+3 = 11 \leq W_{TOT}?$$

YES:

$$\sum_{i \in T} v_i = 7+5+4 = 16 = f$$

NO: $f = 0$

Individual vs. Population

$W_{TOT} = 20$

0	1	1	0	1

INDIVIDUAL

$f = 16$

1	0	1	0	1

$f = 18$

1	1	1	1	1

$f = 0$

0	0	0	0	0

$f = 0$

POPULATION

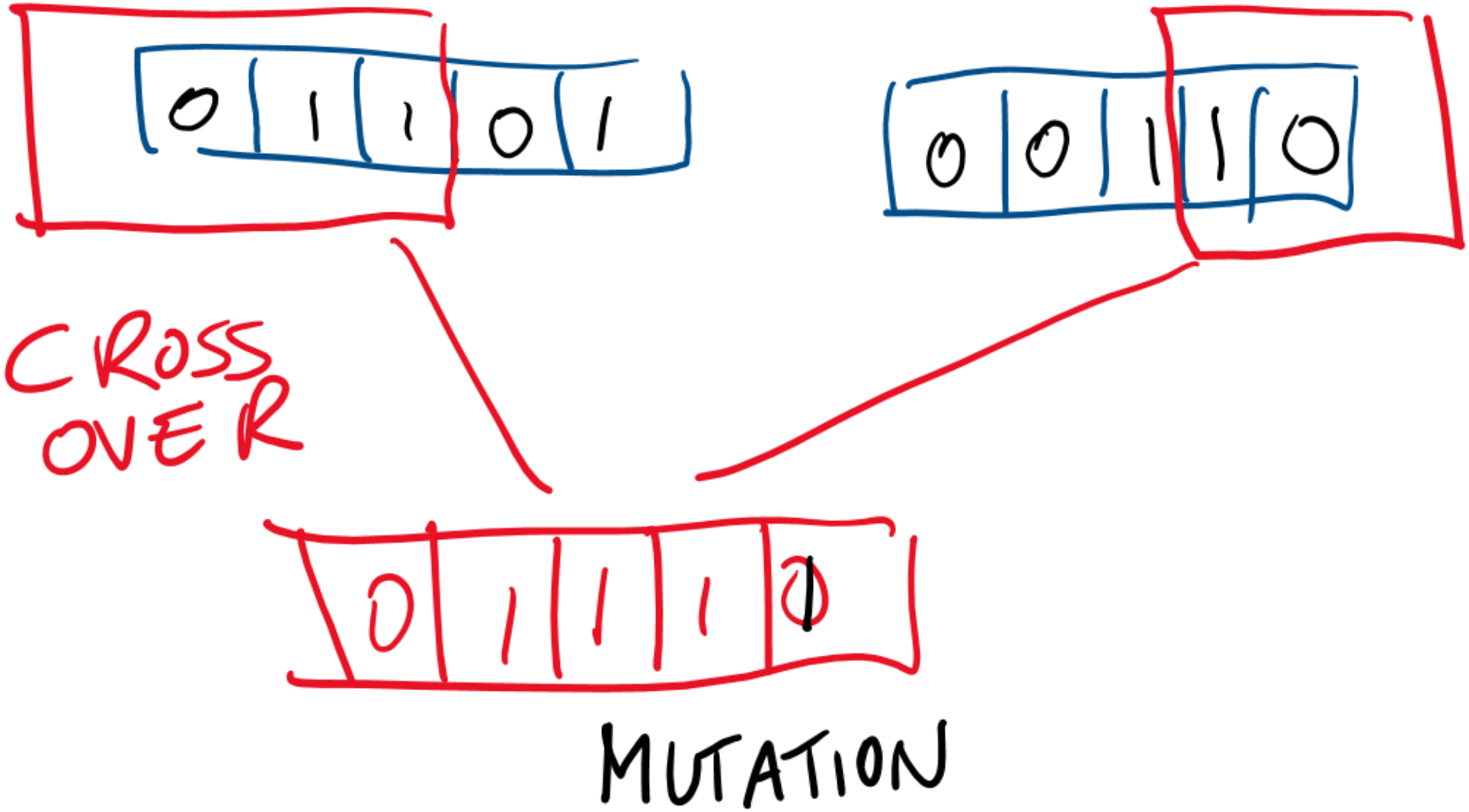
General Evolutionary Algorithm (1)

1. **Initialise** and evaluate **fitness** of **population**

Repeat until quality of solution "**sufficiently good**" (or max generations exceeded):

2. **Select** sub-population for reproduction (**Selection**)
3. **Recombine** the "genes" of selected parents (Recombination or **Crossover**, **XO**)
4. **Mutate** new individuals randomly (**Mutation**)
5. **Evaluate** the **fitness** of the new population
6. **Select** the survivors from the actual fitness

Example



General Evolutionary Algorithm (2)

- Not all these steps are present in all EAs.
- There are as many EAs as researchers in EC!
- We will look at 3 main types:
  - **Genetic algorithms**
  - **Evolution strategies**
  - **Genetic programming**

Genetic Algorithms (GAs)

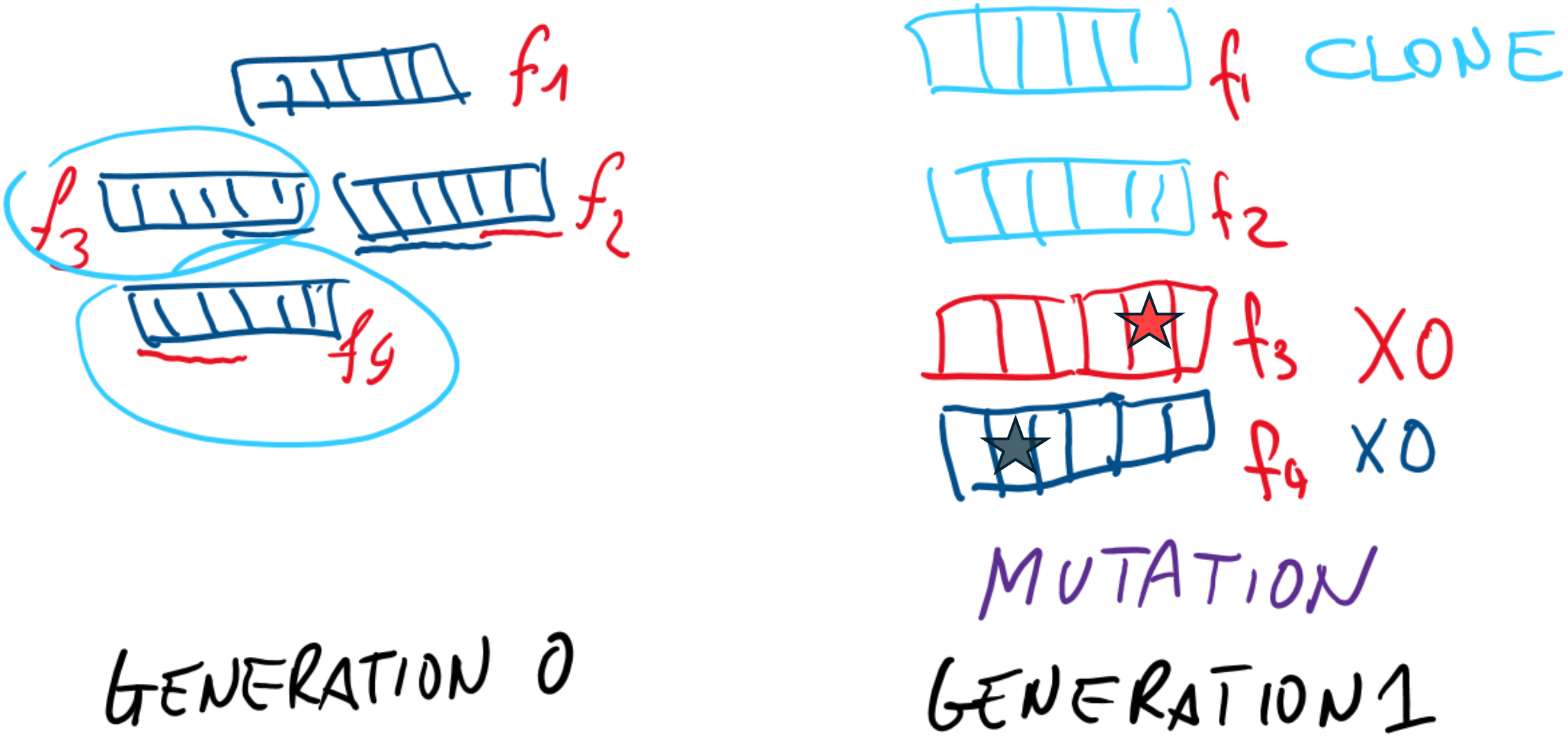
- In GAs there is a strong distinction between **phenotype** (i.e., adult **individual**, **solution**) and **genotype** (i.e., **chromosome**, **representation**)
- GAs (like nature) act on genotypes only and a **genotype-to-phenotype decoding** process is required (i.e., artificial *growth*)
- **Chromosomes** are **strings/arrays of symbols** (originally 0s & 1s)
- Adult individuals can be anything as long as there is a way of representing them using a string of symbols (chromosome).

Generational GA (1)

1. **Randomly generate** a population of chromosomes/individuals
2. **Evaluate** the fitness of each individual (may require **growing/decoding** it first)
3. **Generate** a new population partly by ***cloning*** (copying), partly by ***recombining*** (***crossover***) and partly by ***mutating*** the chromosomes of the fittest individuals
4. Repeat 2 & 3 until a stop condition is true.

Note: There is no overlap between generations (hence **generational GA**). Cloning is used to simulate the survival of parents for more than one generation.

Generational GA (2)



A different example: Problem definition

- **Problem**:

  Find the number in the range $\{0,1,\ldots,255\}$ with the maximum number of 2's in it.

- **Obvious** solution:

  222

A different example: Getting ready

Decisions to be made before starting the GA:

- **Chromosomes**: strings of 8 binary digits (bit strings)
- **Fitness evaluation**: counting the number of 2's in decimal representation of chromosome.
- **Population**: 10 individuals.
- **Selection**: use only the best 5 individuals for mating.
- **Cloning**: each of the selected individuals is copied.
- **Recombination/Crossover**: cutting two bit strings at a random position and swapping the left hand sides.
- **Mutation**: none.

A different example: Initialisation

RANDOM INITIALISATION

#	Genotype	Phenotype	Fitness
a	10010111	151	0
b	01010010	82	1
c	11101010	234	1
d	00101001	41	0
e	11001010	202	2
f	10111110	190	0
g	01001110	78	0
h	10010010	146	0
i	00100000	32	1
j	11111110	254	1

A different example: Selection

SORT

#	Genotype	Phenotype	Fitness
e	11001010	202	2
b	01010010	82	1
c	11101010	234	1
i	00100000	32	1
j	11111110	254	1
d	00101001	41	0
a	10010111	151	0
f	10111110	190	0
g	01001110	78	0
h	10010010	146	0

A different example: Selection

CLONE BEST 5 INDIVIDUALS

#	Genotype	Phenotype	Fitness
e	11001010	202	2
b	01010010	82	1
c	11101010	234	1
i	00100000	32	1
j	11111110	254	1

A different example: Crossover

	#	Genotype	Phenotype	Fitness
Parent	e	1100 1010	202	2
Parent	b	0101 0010	82	1
Offspring	eb	1100 0010	194	0

RANDOM CUT

A different example: Crossover

	#	Genotype	Phenotype	Fitness
Parent	e	11001010	202	2
Parent	c	11101010	234	1
Offspring	ec	11101010	234	1

RANDOM CUT

A different example: Crossover

	#	Genotype	Phenotype	Fitness
Parent	i	00100 000	32	1
Parent	j	11111 110	254	1
Offspring	ij	00100 110	38	0

RANDOM CUT

A different example: Crossover

	#	Genotype	Phenotype	Fitness
Parent	c	11101 010	234	1
Parent	i	00100 000	32	1
Offspring	ci	11101 000	232	2

RANDOM CUT

A different example: Crossover

	#	Genotype	Phenotype	Fitness
Parent	e	110 01010	202	2
Parent	j	111 11110	254	1
Offspring	ej	110 11110	222	3

RANDOM CUT

A different example: New population

#	Genotype	Phenotype	Fitness
e	11001010	202	2
b	01010010	82	1
c	11101010	234	1
i	00100000	32	1
j	11111110	254	1
eb	11000010	194	0
ec	11101010	234	1
ij	00100110	38	0
ci	11101000	232	2
ej	11011110	222	3

INSERT

A different example: New pop. sorted by fitness

SORT

#	Genotype	Phenotype	Fitness
ej	11011110	222	3
ci	11101000	232	2
e	11001010	202	2
b	01010010	82	1
c	11101010	234	1
i	00100000	32	1
j	11111110	254	1
ec	11101010	234	1
eb	11000010	194	0
ij	00100110	38	0

A different example: Observations

- The solution, 222, has been obtained after one generation. Usually, **many generations** are needed to find solutions.
- The **average fitness of the population tends to increase** generation after generation.
- **Good solutions tend to have similar binary strings** (e.g., all solutions end with 0 and the first 3 solutions start with 11…)

Any questions?

Today’s learning objectives (Part 2)

- Learn how to **represent solutions** (integer and real-valued parameters) for **binary Genetic Algorithms** (GAs)
- Become familiar with different ways to **select** individuals for reproduction and **designate** a solution
- Get a clear idea of the difference between **generational** and **steady-state GAs**
- Learn different ways to implement **crossover** and **mutation operators** in binary GA
- Design a **non-binary GA**

Note: least significant bit first!
("Little Indian" bit order)

Solution Encoding in binary GAs

- **Integer parameters**: We can encode integer parameters **p** by using **N** consecutive bits $\mathbf{b_i}$ of a chromosome.
- For instance, $b_1 \ldots b_N$ can be interpreted/decoded as the integer
  $\mathbf{p = b_1 \cdot 2^0 + b_2 \cdot 2^1 + b_3 \cdot 2^2 + \ldots + b_N \cdot 2^{N-1}}$

  E.g., if $\mathbf{b_1 b_2 b_3 b_4 b_5 b_6 b_7 b_8 = 11110000}$, then

  $p = b_1 \cdot 1 + b_2 \cdot 2 + b_3 \cdot 4 + b_4 \cdot 8 + b_5 \cdot 16 + b_6 \cdot 32 + b_7 \cdot 64 + b_8 \cdot 128$

  $= 1 \cdot 1 + 1 \cdot 2 + 1 \cdot 4 + 1 \cdot 8 + 0 \cdot 16 + 0 \cdot 32 + 0 \cdot 64 + 0 \cdot 128 = 1+2+4+8 = 15$

In Python?

```
def decode(chromosome):
    return int(''.join(map(str, chromosome[::-1])),2)
```

Solution Encoding in binary GAs

- **Integer parameters**: We can encode integer parameters **p** by using **N** consecutive bits $\mathbf{b_i}$ of a chromosome.
- For instance, $b_1 \ldots b_N$ can be interpreted/decoded as the integer $\mathbf{p = b_1 \cdot 2^0 + b_2 \cdot 2^1 + b_3 \cdot 2^2 + \ldots + b_N \cdot 2^{N-1}}$ +M

  E.g., if $\mathbf{b_1 b_2 b_3 b_4 b_5 b_6 b_7 b_8 = 11110000}$, then

  $p = b_1 \cdot 1 + b_2 \cdot 2 + b_3 \cdot 4 + b_4 \cdot 8 + b_5 \cdot 16 + b_6 \cdot 32 + b_7 \cdot 64 + b_8 \cdot 128$

  $= 1 \cdot 1 + 1 \cdot 2 + 1 \cdot 4 + 1 \cdot 8 + 0 \cdot 16 + 0 \cdot 32 + 0 \cdot 64 + 0 \cdot 128 = 1+2+4+8 = 15$
- This works well if **p** varies in the range [**0, 1,… 2^N-1**] +M
- What if the smallest **p** must be a number **>0**, say **M**?

Integers (1)

- More generally, if we need to represent only **certain integer values** $[I_1, I_2, \dots, I_N]$ we can use a lookup table

For example, if the only valid values of p are 4, 22, 79, and 311, we can use N=2 and the table

Genotype	Phenotype
00	4
01	22
10	79
11	311

Genotype	Phenotype
00……000	I_1
00……001	I_2
00……011	I_3
…	…
11……111	I_N

Problem: What happens if we have a number of values which isn't a **power of 2**? Why is this a problem?

Integers (2)

Suppose we have L distinct integers $[I_1, I_2, \ldots, I_L]$ and $L \neq 2^N$.

For example, p can be 4, 22, 79, 88, 131 and 311, only. We can use N=3 ➔

Solutions:

- **Clipping**
- **Wrapping**
- **Scaling**

Genotype	Phenotype (CLIPPING)	Phenotype (Wrapping around)	Phenotype (SCALING)
000	4	4	4
001	22	22	4
010	79	79	22
011	88	88	22
100	131	131	79
101	311	311	88
110	311	4	131
111	311	22	311

Real-valued parameters (1)

- **Real-valued parameters** can, <u>in principle</u>, be represented similarly to integer using **a look up table**

- However, if the values are equally spaced, the table can be represented more concisely.

Genotype	Phenotype
00……000	0.1
00……001	0.2
00……010	0.3
00……011	0.4
00……100	0.5
…	…

Real-valued parameters (2)

Fixed-point representation

- If the parameter **p** must be in some **range**, ***[p_{min}, p_{max}]***, and we want to use **N bits** to encode it**,** then we can use:

$$\boldsymbol{p = p_{min} + \left[\frac{decode(b_1, \dots, b_N)}{(2^N - 1)}\right] \cdot (p_{max} - p_{min})}$$

$$\boldsymbol{decode(b_1, \dots, b_N) = b_1 \cdot 2^0 + \ \dots + b_N \cdot 2^{N-1}}$$

- Example: if N=2, p_{min} = -1.0, p_{max} = +1.0 then

$$p = -1.0 + \left[\frac{b_1 \cdot 2^0 + b_2 \cdot 2^1}{(2^2 - 1)}\right] \cdot (+1.0 - (-1.0)) = -1.0 + \left[\frac{b_1 \cdot 1 + b_2 \cdot 2}{3}\right] \cdot 2$$

So, $b_1 b_2$=10 becomes $p = -1.0 + \left[\frac{1 \cdot 1 + 0 \cdot 2}{3}\right] \cdot 2.0 = -1.0 + \frac{1}{3} \cdot 2.0 = -\frac{1}{3}$

Real-valued parameters (3)

- Note: the **resolution**, i.e., the difference between two consecutive reals, is

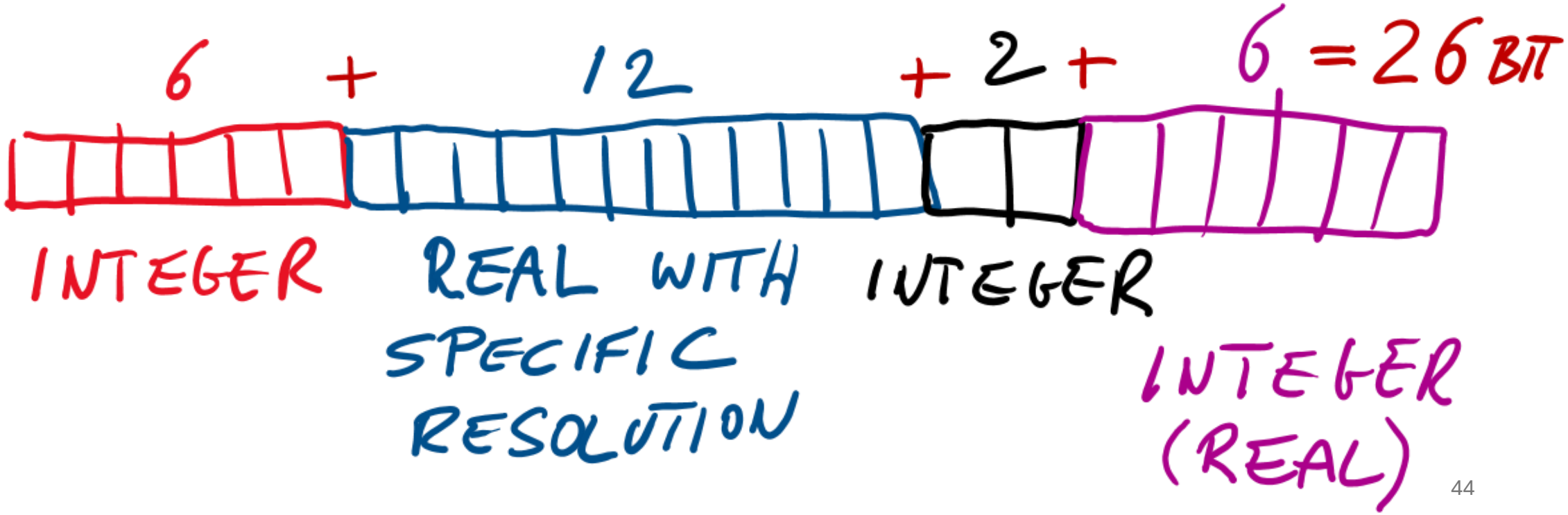
$$\mathbf{resolution} = \frac{p_{max} - p_{min}}{(2^N - 1)}$$

For instance, for N=2 and (p_{max} - p_{min})=2 we have resolution=2/3=0.6666…

So, for real-valued parameters there is a **conflict** between the desire to keep the **genes short** for good convergence and the need to know the result with a certain **precision**.

Solution Encoding in binary Gas (1)

- **Multiple of parameters** (whether binary, integers, reals or a combination of them) are encoded combining (e.g., concatenating) their representations



Solution Encoding in binary Gas (2)

- Concatenation is not the only method
- When the same number of bits is used to encode each parameter, **interleaving** them can also be good under 1-point crossover
- If b_i^p = bit i of parameter p
  - *Traditional:* $\underbrace{b_1^1\, b_2^1\, b_3^1 \ldots}_{P_1}\underbrace{b_1^2\, b_2^2\, b_3^2 \ldots}_{P_2}\ldots\underbrace{b_1^N\, b_2^N\, b_3^N \ldots}_{P_N}$
  - *Interleaved:* $b_1^1\, b_1^2\, b_1^3 \ldots b_2^1\, b_2^2\, b_2^3 \ldots b_N^1\, b_N^2\, b_N^3 \ldots$

P_1 P_2 .... P_N

P_1 P_2 ....

Selection basics (1)

- **Selection** is the operation by which individuals (chromosomes) are randomly selected **for mating** or other genetic operations (mutation and cloning).
- Obviously for **crossover**, *pairs* of individuals are needed to produce an offspring
- For **cloning**, only single individuals are picked by each selection invocation.
- To emulate natural selection **individuals with a higher fitness should be selected with a higher probability**
- There are many different models of selection: some that are *biologically plausible* and some that are not.

Selection basics (2)

- Assumptions (for now):
  - A **quality measure Q** (objective function) for the solutions of the problem is **known**.
  - Q has to be **maximised**.
  - Q is (almost) **always positive**.
  - We take the (raw) **fitness** *f* of an individual to be its quality measure Q, i.e.,

$$f = Q$$

  - For instance, if Q=total value of knapsack we can use $f = Q$

Fitness Proportionate Selection (FPS)

- FPS is often used in GAs.
- Let $\boldsymbol{f_i}$ be the **fitness** of individual *i*, $\boldsymbol{f_{avg}} = \frac{f_1+f_2+\cdots+f_M}{M}$ be the **average fitness** and *M* be the **size of the population**
- In FPS, individual *i* is selected for reproduction with a **probability**

$$\boldsymbol{p_i} = \frac{f_i}{f_1 + f_2 + \cdots + f_M} = \frac{f_i}{f_{avg} \cdot M} = \frac{f_i}{f_{avg}} \cdot \frac{1}{M}$$

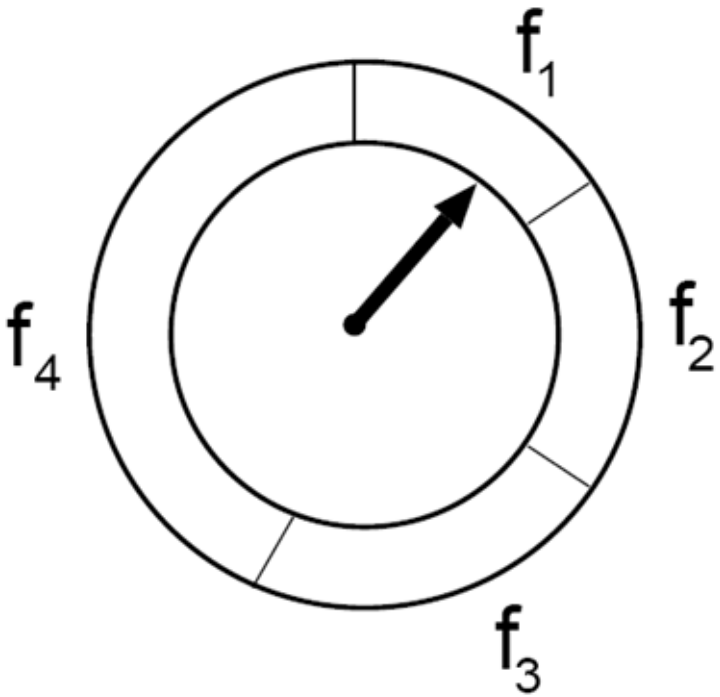
Roulette-wheel algorithm (1)

- For example, if M = 4, $f_1 = f_2 = 10$, $f_3 = 15$ and $f_4 = 25$

INCLUDED

- **Implementation**: a random number r in $[0, f_1 + f_2 + \ldots f_M)$ EXCLUDED is chosen and individual i is selected if

$$f_1 + f_2 + \ldots + f_{i-1} \leq r < f_1 + f_2 + \ldots + f_i$$



Roulette-wheel algorithm (1)

- In the example, $f_1 + f_2 + f_3 + f_4 = 60$, so r is a random number in [0,60)
- Suppose the random number generator produces r = 23.5, then ➔

i	Test	Result
1	0 <= 23.5 < 10	False
2	10 <= 23.5 < 20	False
3	20 <= 23.5 < 35	True
4	35 <= 23.5 < 60	False (but unnecessary)

f_1

f_1+f_2

$f_1+f_2+f_3$

So, i=3.

Properties of FPS (1)

- Imagine that **only cloning is taking place** (no XO, no Mut) to produce a new generation of *M* individuals.
- Because we use cloning, *M* individuals have to be selected.
- So, the **expected number of copies** of each individual in the new generation is: $N_i = p_i \cdot M = \frac{f_i}{f_{avg}}$
- N_i is usually a real number. The actual number of copies (an integer) will vary around N_i.

Properties of FPS (2)

- Since $N_i = \frac{f_i}{f_{avg}}$, individuals with $\boldsymbol{f_i} > \boldsymbol{f_{avg}}$ tend to have **more than one copy** in the next generation
- For the same reason, **below average individuals tend not to be copied**.
- In a GA using *recombination* (crossover) and mutation, the principle is the same:

  Above average individuals have a higher-than-average probability of being chosen as parents and vice versa.

Problems with FPS: Premature Convergence

- Suppose an individual with f_i *much bigger than* f_{avg} but f_i much smaller than f_{max} is produced in an early generation.
- Such an individual will be *selected very often* for crossover and cloning.
- So, unavoidably the **genes** of such individual **quickly spread** all **over** the **population, rendering it too uniform**.
- At that point *recombination cannot generate any new solutions* (only mutation can) and f_{avg} is likely to remain much smaller than f_{max} forever.

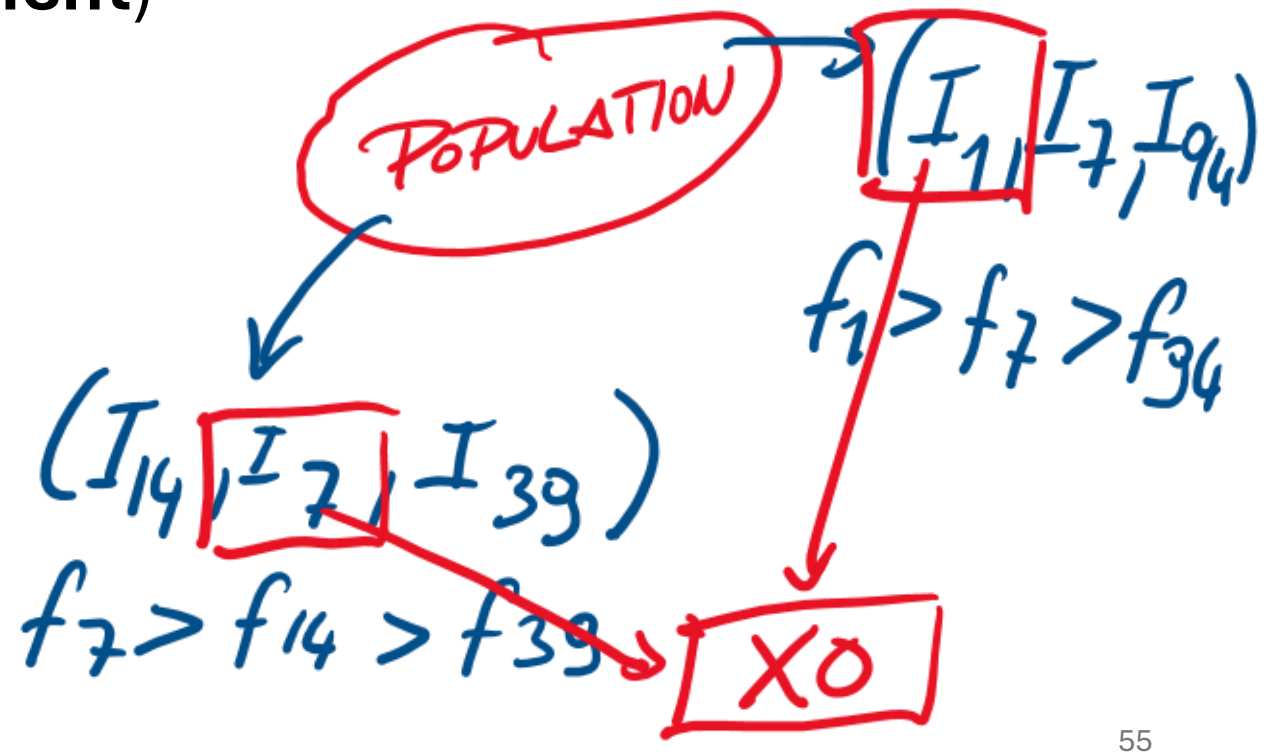
Problems with FPS: Stagnation

- Towards the *end of a run* all individuals tend to have relatively high and similar values of fitness, i.e., f_i is approximately equal to f_{max} for all *i*
- At that point N_1 … N_M are all approximately 1, so there is a very **small selective pressure**, i.e., the best solutions are favoured only slightly with respect to the worst ones.

Both problems can be solved with *fitness scaling* techniques, but **other selection** methods avoid the problems more naturally.

Tournament selection

- A group of ***N* ≥ 2 random** individuals is created
- The individual with the highest fitness in the group is selected, the others are discarded (**tournament**)
- Advantages:
  - No premature convergence
  - No stagnation
  - No global reordering required
  - Explicit fitness not needed
  - Naturally inspired



Solution designation

Which solution (individual) among the candidate solutions to chose?

- **Fittest individual** in the **last** generation
- Alternatively: save the **fittest solution ever discovered**, designate such structure as the solution

Steady State GAs

- In **steady state GAs**, the offspring are **inserted** in the **current population**, usually **replacing** the **worst individual** in the population.
- Sometimes an **inverse tournament** selection is used and the worst in the tournament is replaced.
- Steady state GAs require less memory and **may be more efficient** as they exploit the genetic material of a good solution immediately.

Properties of Crossover Operators

- Evolutionary algorithms work well only if their genetic operators allow an **efficient and effective search of the solution space**.
- Typical properties of **good genetic operators**:
  - When parents share a **common feature** their offspring should always inherit such a feature.
  - When parents have two **different features** it should be possible for the offspring to inherit both such features.
- These properties are often *partially incompatible* and operators with such properties can be more computationally demanding.

Crossover in binary GAs (1)

- **One-point crossover** involves cutting the chromosomes of the parents at a randomly chosen common point and exchanging the right-hand-side sub-chromosomes

Parent a	11	111111
Parent b	00	000000
Offspring	11	000000

CUT

Crossover in binary GAs (2)

- **Two-point crossover** involves cutting the chromosomes of the parents at two randomly chosen common points and exchanging the middle sections

Parent a	11	111	111
Parent b	00	000	000
Offspring	11	000	111

- **N-point crossover** is similarly defined.

Crossover in binary GAs (3)

- In **uniform crossover** each gene of the offspring is selected **randomly** from the corresponding genes of the parents, e.g.

Parent a	11111111
Parent b	00000000
Offspring	11010110

- **Crossover** is applied to the individuals of a population with a **constant probability/rate p_c** (typically **p_c is within [0.5, 0.8]**)
- **Cloning** is applied with a probability **1- p_c** to keep the size of the population constant

Mutation in binary GAs (1)

- **Mutation** consists of making (usually small) alterations to the values of one or more genes in a chromosome
- In GAs, traditionally mutation is applied to the individuals produced by crossover and cloning **before they are added to the population**.

Mutation in binary GAs (2)

- In binary chromosomes mutation consists of **inverting random bits (bit flip)** of the genotypes, e.g.

Parent	1111111
Offspring	1110110

- Traditionally, in GAs, mutation is considered a method to **recover lost genetic** material (rather than to search for better solutions)
- In GAs, mutation is applied with a **very low probability/rate p_m** (per bit). Generally p_m is within **[0.001, 0.01]**.
- **A good rule of thumb is p_m =1/Number of bits**

Non-binary GAs (1)

- Non-binary chromosomes can be used to encode parameters that can take only a small number of values.

- Examples:
  - If a parameter p is in the set $\{0, 1, 2\}$ we can simply use a **ternary alphabet** instead of a binary one
  - If p is in the set {Rome, Milan, Birmingham, London, New York, Paris}, we can encode it using letters {R,M,B,L,N,P} or numbers {0,…,5}

Non-binary GAs (2)

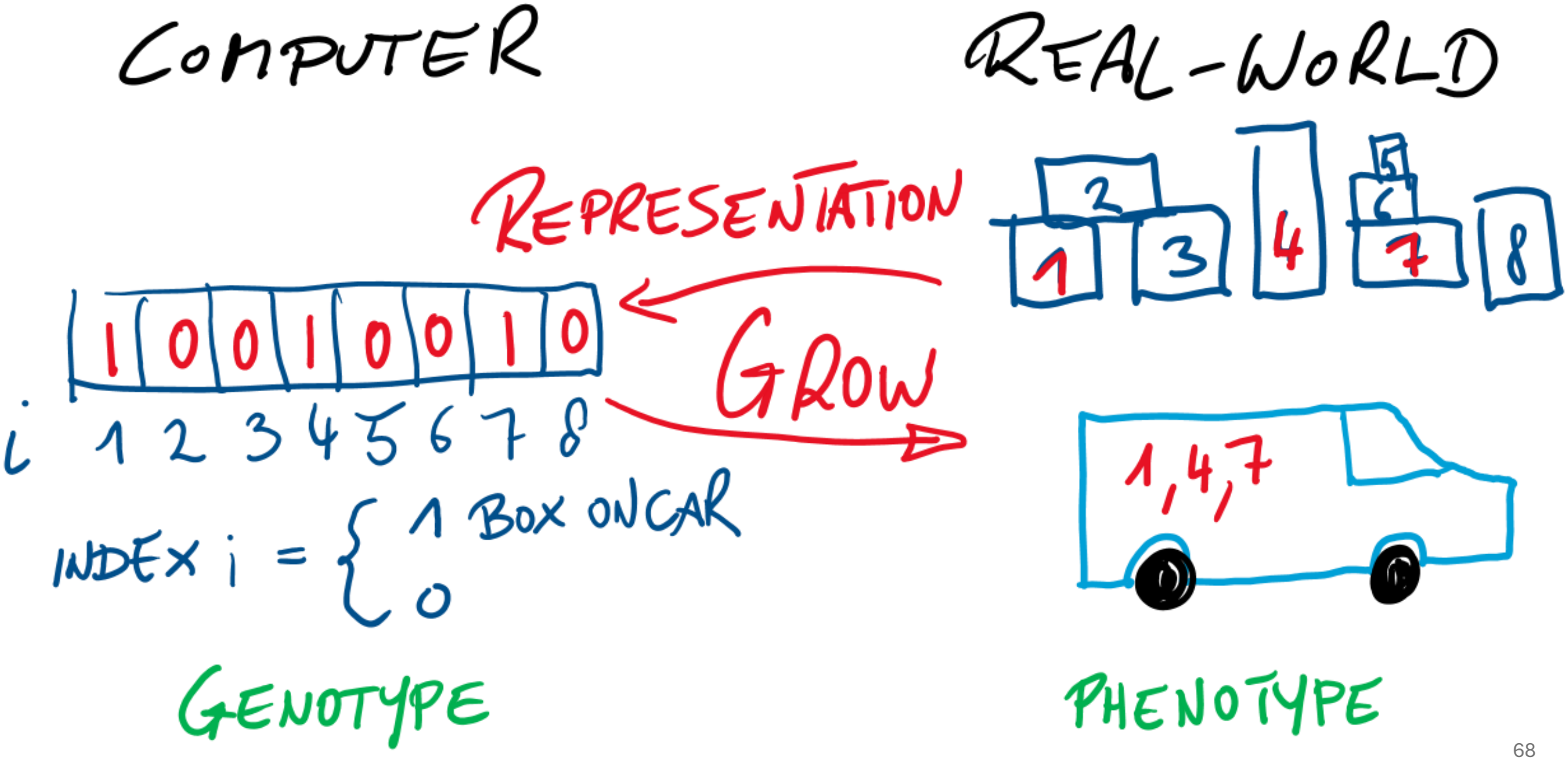
- Genes may be treated as atomic (discrete) or not based on the application domain.
- If a gene is 3.2 and another is 3.4, does it make sense to talk about a gene half-way between them (3.3)? What about 36 and 42 vs 39?
  - If 3.2 and 3.4 represent continuous quantities, e.g., temperatures, then 3.3 make senses. If they represented version numbers of a public API, maybe 3.3 was never released!
  - If 36 and 42 represent number of apples, then 39 is a valid outcome, but if 36=Colchester and 42=Johannesburg, what is 39?
- **Different forms of crossover and mutation are needed depending on whether it makes sense to interpolate genes or not.**

Any questions?

What did we learn today? (1)

- We have learned how basic concepts of evolution in nature can be applied to solve real world problems.
- We learned about the **General Evolutionary Algorithm (GEA)**, which iteratively applies operators such as **selection, cloning, recombination**, and **mutation** to evolve solutions that are “fitter” and provide a more satisfactory solution to the problem.
- If we can find a linear representation for the solution of a problem, then we can use the GEA approach to solve that problem.

What did we learn today? (2)



What did we learn today? (2)

- We have learned how to encode **integer** and **real-valued parameters** for **binary GA** and that for real-values there is a **trade-off** between **bitstring length** and **precision**.
- We have learned about the concepts of **fitness proportionate selection (FPS)** and **tournament** selection.
- We have learned that **steady-state GAs** insert offspring into the current population and exploit the genetic material of good solutions immediately.

What did we learn today? (3)

- We learned about **one-point**, **two-point** and **uniform crossover** (**XO**) and that these methods are applied to individuals with a certain **probability p_c** with **cloning** being applied with a probability of **$1- p_c$**.
- We have learned that **mutation** is applied **after** crossover or cloning with a **very low probability per bit**.
- We have learned that GAs can be used with **non-binary** alphabets.

[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Thank you for your participation

Next time:
GA applications and advanced techniques!

University of Essex
School of Computer Science and Electronic Engineering